

Personalized E-Learning Dashboard – Project Binder

This binder captures the working requirements, architecture notes, testing strategy, and maintenance procedures for the Personalized E-Learning Dashboard. It should accompany every release/deployment so future maintainers can quickly understand expectations and workflows.

1. Product Overview

The dashboard delivers curated learning paths for technical skills (coding, SQL, data analysis) with adaptive quizzes, gamified streak/badge tracking, and a threaded forum for collaboration. Scope is intentionally limited to ~10–15 modules to keep content manageable while still demonstrating personalization and progression.

Key differentiators:

- Adaptive quiz engine that adjusts difficulty based on previous answers.
 - Streak tracker + tiered badges (Bronze → Ruby) visualizing progress.
 - Course-specific forum threads with nested replies.
 - Responsive UI targeted for both desktop and mobile devices.
-

2. Requirements

Functional Requirements

1. **Authentication:** Users can sign up/log in, and sessions persist per user.
2. **Dashboard:** Displays module progress, streak metrics, and earned badges.
3. **Adaptive Quizzes:** Each subject delivers 5 questions per session, automatically adjusting difficulty (easy/medium/hard) per answer.
4. **Question Pools:** Every subject maintains at least 15 questions (5 per difficulty level).
5. **Forum:** Users can post and reply within course-specific threads; all users see the full discussion tree.
6. **Badges & Streaks:** Progress generates Bronze/Silver/Gold/Platinum/Ruby badges plus streak visualizations.
7. **Health Endpoint:** `/health` responds 200 for platform health checks.

Non-Functional Requirements

- **Responsiveness:** Layout adapts to tablets/phones; sidebar collapses to a drawer under 992px.
 - **Availability:** Backend health check must respond within platform timeouts (Railway).
 - **Maintainability:** Configuration via environment variables (`REACT_APP_API_BASE`, `DATABASE_URL`, etc.).
 - **Security:** CORS restricted to deployed frontend domain; password hashing via Werkzeug.
-

3. Architecture & Deployment

Components

- **Frontend:** React (Create React App) deployed on Vercel.

- **Backend:** Flask + SQLAlchemy deployed on Railway (Python 3.11), SQLite persisted via mounted volume (`/app/data/db.sqlite`).
- **State:** SQLite database storing users, modules, quizzes, questions, progress, forum posts, badges.

Environment Variables

- `REACT_APP_API_BASE` (frontend) → Base URL to backend (e.g., `https://elearningdashboardnew-production.up.railway.app`).
- `DATABASE_URL` (optional) → Use if migrating to hosted DB.
- `PERSISTENT_DB_PATH` (backend) → Path to SQLite file on Railway volume (`/app/data/db.sqlite`).
- `FLASK_ENV`, `SECRET_KEY`, `PORT` (Railway auto-sets `PORT`).

Deployment Workflow

1. Backend (Railway):

- Set `PYTHON_VERSION=3.11`.
- Build command `pip install -r backend/requirements.txt`.
- Start command `python app.py`.
- Mount volume at `/app/data`.
- Health check path `/health`.

2. Frontend (Vercel):

- Root directory `frontend`.
- Build `npm run build`.
- Output `build`.
- Env var `REACT_APP_API_BASE` set to backend URL.

3. Post-deploy validation:

- Hit `/health` on backend.
- Visit frontend URL, log in, run quiz, post on forum.

4. Maintenance Procedures

Task	Frequency	Steps
Database backup	Weekly	Download <code>/app/data/db.sqlite</code> from Railway or run <code>sqlite3 .backup</code> .
Dependency updates	Monthly	<code>pip list --outdated</code> , <code>npm outdated</code> ; update versions & redeploy.
Log review	Weekly or after incidents	Inspect Railway logs + Vercel analytics for errors.
Security checks	Quarterly	Verify CORS origins, rotate <code>SECRET_KEY</code> , confirm TLS endpoints.

Task	Frequency	Steps
Content refresh	Quarterly	Add/adjust module descriptions, quiz questions, forum moderation policies.

5. Testing Strategy

Automated / Manual Regression

- **API Smoke Tests:** Manual hit `/health`, `/quizzes`, `/dashboard/<id>`, `/forum`.
- **Frontend Regression:** Verify login, dashboard, quiz flow, forum post/reply.
- **Responsive Snapshot:** Use Chrome DevTools (iPhone X, iPad) to confirm layout.

Mock Learner Study (15 Participants)

1. **Participant Pool:** Seed 15 mock learners with varying skill levels (insert via script or admin page).
2. **Pre-Test Survey:** Capture baseline engagement (Likert scale on motivation/confidence).
3. **Usage Session:** Each learner completes:
 - One quiz per subject (5 questions) capturing score + streak.
 - At least one forum post.
4. **Post-Test Survey:** Same questions + qualitative feedback.
5. **Metrics to Record:**
 - Score improvements per module.
 - Time to complete quiz.
 - Number of forum interactions.
 - Self-reported engagement change.
6. **Reporting:** Summarize in a Confluence/Markdown report with charts (bar chart for pre vs post engagement, average streak growth).

6. Module Catalog (Snapshot)

Module	Category	Description	Duration (min)
Python Foundations	Programming	Variables, control flow, functions	60
Advanced Python OOP	Programming	Classes, inheritance, design patterns	75
SQL Basics	Data	SELECT/WHERE, filtering, aggregation	50
SQL Joins Mastery	Data	INNER/LEFT/RIGHT joins, set operations	60
Data Visualization	Data	Matplotlib/Seaborn charting	45
Data Structures	CS	Arrays, stacks, queues, trees	60
Algorithms 101	CS	Sorting/searching, Big-O intuition	70
Web Design with JavaScript	Frontend	DOM manipulation, event handling	55
React Essentials	Frontend	Components, props, hooks	65

Module	Category	Description	Duration (min)
Cloud Computing	Cloud	IaaS/PaaS/SaaS, AWS/Azure basics	60
DevOps Fundamentals	DevOps	CI/CD, containers, monitoring	70
Machine Learning Intro	AI	Regression/classification overview	65
Deep Learning Basics	AI	Neural nets, CNN/RNN basics	75
Data Ethics & Privacy	Governance	Responsible data usage	40
Agile Collaboration	Soft Skills	Scrum rituals, retrospectives	35

Module data should be mirrored in the database seed scripts to meet the 10–15 module requirement.

7. Future Enhancements

- Swap SQLite for managed Postgres (Railway/Neon) for scalability.
- Add in-app notifications for streak milestones or forum replies.
- Integrate analytics dashboards (Mixpanel/Amplitude) for real engagement data.
- Automate user testing surveys with embedded forms + metrics pipeline.

8. Quick Reference / README Link

- A concise setup and deployment guide lives in `README.md` at the repo root. It covers local commands, env vars (`REACT_APP_API_BASE`, `SQLALCHEMY_DATABASE_URI`), seeding from `backend/question_seed_data.py`, and the `/health` check.
- Frontend reads `REACT_APP_API_BASE`; quizzes are gated to modules the learner selects.
- Backend seeds questions on startup; keep `backend/question_seed_data.py` current with module/content updates.

Contact / Ownership

- **Suomayh's_Group** Suomayh's_Group
- **Technical Owner:** cK
- **Support:** Use GitHub issues or Slack channel `#elearning-dashboard`.